Sprint 2 Contribution Log

SOEN 341: CareerConnect

Youssef Karnib, General Backend Developer

Overview: In Sprint 2, I worked on the FastAPI backend for job listing and search (YO-1), recruiter job posting and application viewing (YO-3), and application status tracking (YO-2). I reused the team's existing database queries, authentication and application submission functionality, added API schemas and access checks, and created an Alembic migration for application timestamps and statuses. I tested the features using Swagger UI and ran the team's automated tests.

Time spent: The personal sprint guide estimated 4 hours for YO-1, 4 hours for YO-3 and 3.5 hours for YO-2 (11.5 hours of planned feature work). Actual session durations were not recorded in the available activity history; the work log below records verified activities rather than claiming unverified hours.

YO-1: Job Listing, Detail and Search Endpoints

Goal: Allow an authenticated user to list open jobs, search and filter them, and view complete details for a selected job.

What I did:

- Created response schemas for job list items, full job details and the list response (items and total) in App/schemas/job.py.
- Added GET /api/v1/jobs with keyword, location, category and page parameters, and GET /api/v1/jobs/{id} with a 404 response for missing jobs (App/routers/jobs.py).
- Reused Kaila's existing search_jobs and get_job CRUD functions; job listings only include open jobs and omit long descriptions.
- Added token-based current-user and recruiter authorization helpers during initial development (App/core/dependencies.py); registered the jobs router in App/main.py and adjusted validation handling to preserve Sprint 1 registration behavior.

Verification: Tested in Swagger UI by registering and logging in, authorizing with a token, listing jobs, viewing the newly created posting and checking filters and job details. At the initial submission stage, 17 backend tests and 1 frontend sanity test passed.

Time: Personal guide allocation: about 4 hours (actual elapsed time not tracked).

Links:

- Commit e22f708: YO-1 authenticated job listing, search and details.
- Pull request #102 (merged October 26, 2026).

YO-3: Recruiter Job Posting and Applicant Endpoints

Goal: Let a recruiter create a job posting and view applications received for their own posting.

What I did:

- Created the posting request schema with required fields and the six allowed categories, plus the applicant-list response schema (App/schemas/recruiter.py).

- Implemented POST /api/v1/recruiter/jobs (201 Created) and GET /api/v1/recruiter/jobs/{id}/applications (App/routers/recruiter.py).
- Restricted endpoints to recruiters, assigned each posting to the logged-in recruiter, and returned 404 when a recruiter tried to view applicants for a posting they did not own.
- Returned the applicant's profile full name when available, falling back to the user email; integrated the recruiter router in App/main.py.

Verification: Created a recruiter test account and a sample job in Swagger UI; confirmed that it appeared in the job listing and that a new posting had an empty applications list. Ran the existing backend and frontend test suites during this work.

Time: Personal guide allocation: about 4 hours (actual elapsed time not tracked).

Links:

- Pull request #103 (merged into the YO-1 feature branch on October 26, 2026; integrated with YO-1).

YO-2: Application Status Tracking and Updates

Goal: Allow users to view their submitted applications and allow the recruiter who owns a job to change an application's status.

What I did:

- Implemented GET /api/v1/applications/me, returning only the authenticated user's applications, newest first, with job title, company, status, applied_at and updated_at.
- Implemented PATCH /api/v1/applications/{id}/status, requiring recruiter permissions and checking ownership of the job associated with the application (App/routers/application_status.py).
- Used the five official statuses: Applied, Under Review, Interview, Offered and Rejected; added request and response schemas to App/schemas/application.py.
- Updated App/models/application.py to add updated_at and use Applied as the default status. Created Alembic revision 16d4c4e691bf to initialize historical timestamps and update the database default.
- Registered the new router in App/main.py and adjusted a recruiter test that still expected the obsolete submitted status.

Verification: Applied the migration successfully to the local SQLite database. In Swagger UI, uploaded a test resume, submitted an application as a job seeker, changed its status to Interview as the owning recruiter, and verified the new status in My Applications. After updating the outdated assertion, 72 backend tests passed.

Time: Personal guide allocation: about 3.5 hours (actual elapsed time not tracked).

Links:

- YO-2 branch: feature/YO-2-application-status; migration 16d4c4e691bf.
- Repository: SOEN341-2026. A YO-2 commit or pull request URL was not confirmed in the available record and should be added after submission.

Work Log

October 23, 2026: Reopened the Sprint 1 clone in VS Code, synchronized branches, built and tested YO-1 and YO-3 endpoints, created test users and a job posting through Swagger UI, fixed regression in validation error handling, ran 17 backend and 1 frontend tests, and pushed the feature work. PRs #102 and #103 were opened.

October 26, 2026: Updated the local repository after Lamees's application endpoint became available. Built YO-2 response models and endpoints, synchronized local Alembic state, added the updated_at/status-default migration, fixed a missing local python-multipart dependency, tested the Applied-to-Interview workflow, and reached 72 passing backend tests.

AI Usage and Validation

Used ChatGPT for step-by-step development guidance, code drafts, Git and Alembic commands, debugging, and Swagger test procedures. Reviewed generated changes against the team's existing models and routes, tested the behavior in Swagger UI, and ran automated tests to validate integration. A separate AI usage log should record prompts, adopted code and verification details as required by the course.

Record note: Feature descriptions, test results and dated activities are based on the available development record. Planned-hour allocations are not reported as measured time.